

Performance Testing

Date	1 july 2026
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the OptiCrop system behaves under expected and peak load conditions. The sections below document the testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based open-source load testing tool)
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	/predict endpoint (Crop Optimization API) – accepts soil & climate parameters and returns ML-based crop recommendation
Test Environment	Hosted Server – Render (opticrop.onrender.com)
Test Date	1 july 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Load – Simulate typical concurrent farmer users submitting soil/climate data for crop recommendation	50	60	All requests processed; response < 3 sec; 0% error rate
2	Peak Load – Simulate high concurrent usage during planting season demand surge	200	60	System handles load; avg response < 5 sec; error rate < 2%
3	Stress Test – Gradually ramp users beyond normal capacity to find breaking point	500	120	System degrades gracefully; no crash; error rate noted and logged

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
4	Spike Test – Sudden burst of simultaneous requests simulating viral traffic	300 (sudden spike)	30	System recovers within 10 sec after spike; no data corruption

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 sec	Pass	Consistent under normal load
2	Response Time (Max)	< 5 seconds	4.1 sec	Pass	Observed during spike test
3	Throughput (Req/sec)	–	38 req/sec	Pass	Stable under 200 users
4	Error Rate	< 1%	0.3%	Pass	Minimal errors at peak load
5	CPU Utilization	< 80%	62%	Pass	Flask + ML inference within limits
6	Memory Utilization	< 80%	58%	Pass	No memory leaks observed

Step 4: Observations & Analysis

Key Findings:

OptiCrop's /predict endpoint performed well under normal and peak load conditions. Average response time of 1.4 seconds is well within the 2-second target. The Scikit-learn ML inference (multi-algorithm pipeline: Logistic Regression, KNN, Decision Tree, Random Forest, K-Means) proved to be lightweight and fast on the hosted Render server. Throughput reached 38 req/sec with 200 concurrent users without significant degradation.

Bottlenecks Identified:

1. Cold start latency on Render free tier: the first request after inactivity (server spin-up) takes 8-12 seconds; subsequent requests are fast. 2. Under the 500-user stress test, response time increased to ~4.1 sec (still within the 5-sec target), and error rate rose to 0.3% due to occasional request queuing on the free-tier Render instance. 3. The ML pipeline loading (model pickle files) contributes ~0.3 sec to each response; this could be optimized by caching loaded models at server startup.

Optimization Steps Taken:

1. ML models are now loaded once at application startup (cached in memory) rather than reloaded per request, reducing per-request latency by ~0.3 sec. 2. Input validation (checking N-P-K, temperature, humidity, pH, rainfall are numerical and non-negative) is performed before invoking the ML pipeline, avoiding unnecessary compute on bad inputs. 3. Render deployment configured with gunicorn workers to handle concurrent requests more efficiently than Flask's built-in dev server.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., Locust graphs, response time charts):

Screenshot 1: Locust Load Test – Request Statistics Report

Shows RPS (requests per second), average/min/max response times, and failure count for the /predict endpoint under 200 virtual users over 60 seconds. All key metrics within target thresholds.

Screenshot 2: Locust Charts – Response Time & Users Over Time

Line chart showing response time (median and 95th percentile) and number of users over the test duration. Confirms stable performance with no major spikes under normal and peak load scenarios.